

Backend API Architecture Specification

This document provides a comprehensive technical blueprint for the Backend Development team (e.g., Node.js, Express, MongoDB). It defines the RESTful APIs, WebSockets, and business logic required to act as the common engine for both the Mobile App (A01-A43) and Web Portal (W01-W43). **(Updated Version v2.0)**

1. Authentication & Users (/api/users)

API-01: User Login & Auth

- **Method:** POST /api/users/login
- **Models:** UserModel
- **Logic:** Authenticate via Email/Password (bcrypt) or Google OAuth. On success, generates a JWT token embedding `_id` and `role_id`.
- **Supports Frontend:** A31, W31

API-02: Get User Profile & Dashboard Stats

- **Method:** GET /api/users/profile
- **Models:** UserModel, TaskModel, LeaveApplicationModel, RoleModel
- **Logic:** Decodes the JWT. Returns user details, user's RoleModel and policies (for UI RBAC), task_count, active tasks, and organization-wide stats (total active members, members currently on leave).
- **Supports Frontend:** A01, A02, A05, W01, W02, W05, A27, W27

API-03: Team Directory & Global Search

- **Method:** GET /api/users/directory & GET /api/search

- **Models:** `UserModel` (with aggregation on `TaskModel`, `MeetingModel` for global search)
- **Logic:** Directory supports filtering by `batch` and `domain`. Global search implements text-index searching across multiple collections to return categorized results.
- **Supports Frontend:** A07, A35, W07, W35

API-04: Forgot / Reset Password

- **Method:** `POST /api/users/forgot-password` & `POST /api/users/reset-password`
- **Models:** `UserModel`
- **Logic:**
 - *Forgot:* Generates a secure OTP/Token, saves it in the database under `UserModel.reset_token` along with an expiry timestamp, and sends it via Email service.
 - *Reset:* Verifies the submitted token against the database, and securely hashes the new password upon verification.
- **Supports Frontend:** A36, W36

API-05: Bulk Member Upload (Admin)

- **Method:** `POST /api/users/bulk-upload`
- **Models:** `UserModel`
- **Logic:** Requires RBAC Admin Middleware. Parses uploaded CSV, hashes default passwords (or triggers invite emails), and bulk inserts records using `insertMany`.
- **Supports Frontend:** A41, W41

API-06: Member Management (Admin)

- **Method:** `GET`, `PUT /api/users/:id`
- **Models:** `UserModel`
- **Logic:** Admins fetch all users for the management dashboard. PUT updates `status` (make alumni), `volatile_data`, or `role_id`.

- **Supports Frontend:** A38, W38

2. Roles & IAM RBAC (/api/iam)

API-07: Manage Roles & Policies (SuperAdmin)

- **Method:** GET , POST , PUT /api/iam/roles
- **Models:** RoleModel , PolicyModel
- **Logic:** CRUD operations for dynamic roles.
- **Middleware Note:** A global RBAC Middleware must sit in front of all protected routes to evaluate PolicyModel.statements against the requester's role_id before allowing the request to proceed.
- **Supports Frontend:** A40, W40

3. Tasks Management (/api/tasks)

API-08: Fetch Tasks (List & Calendar)

- **Method:** GET /api/tasks
- **Models:** TaskModel
- **Logic:** Returns tasks filtered by status , assigned_to , or date ranges (for rendering the calendar). Includes pagination.
- **Supports Frontend:** A04, A08, A09, A11, A19, W04, W08, W09, W11, W19

API-09: Create & Assign Task (Admin)

- **Method:** POST , PUT /api/tasks/:id
- **Models:** TaskModel , NotificationModel
- **Logic:** Validates payload (priority, attachments, domain_id). Saving or updating the assigned_to array automatically triggers asynchronous creation of

`NotificationModel` records. Logs all changes to the `history` array for the Audit Trail.

- **Supports Frontend:** A10, A13, W10, W13

API-10: Task Workflow (Accept/Submit/Review)

- **Method:** `POST /api/tasks/:id/action`
- **Models:** `TaskModel`
- **Logic:** Endpoint to handle 'accept', 'submit', and 'review' actions. Appends JSON data to `submissions` or `reviews` arrays and updates `status`.
- **Supports Frontend:** A12, W12

4. Meetings & Attendance (`/api/meetings`)

API-11: Meeting Dashboard & Creation

- **Method:** `GET` , `POST /api/meetings`
- **Models:** `MeetingModel`
- **Logic:**
 - *POST:* Create meeting validates RBAC. Requires mapping the creator's user ID to `created_by`.
 - *GET:* Fetches meetings filtered by `status` (Upcoming/Ongoing/Completed). Crucially, the query MUST filter by `target_batch` and `target_domain` against the requesting user's profile to only return meetings that the user is authorized/targeted to see.
- **Supports Frontend:** A03, A14, A15, A16, W03, W14, W15, W16

API-12: Attendance & MOM Control

- **Method:** `GET` , `PUT /api/meetings/:id/attendance`
- **Models:** `MeetingModel` , `AttendanceModel`
- **Logic:** PUT: Admins update `mom_content` and batch update `AttendanceModel`

records (`is_present` , `reason_if_absent`). GET: Aggregates attendance statistics for registry tables.

- **Supports Frontend:** A17, A18, A20, W17, W18, W20

5. Forms & Applications (`/api/forms` & `/api/leaves`)

API-13: Forms & Submissions

- **Method:** `GET` , `POST /api/forms` , `POST /api/forms/:id/submit` , `GET /api/forms/:id/submissions`
- **Models:** `FormModel` , `FormSubmissionModel`
- **Logic:** GET fetches active forms. Submit route validates JSON against `fields` . The submissions GET route fetches the aggregated `responses` JSON for Admin Dashboards.
- **Supports Frontend:** A23, A24, A25, A26, W23, W24, W25, W26

API-14: Leave Applications

- **Method:** `GET` , `POST` , `PUT /api/leaves`
- **Models:** `LeaveApplicationModel` , `NotificationModel`
- **Logic:** Users POST new leaves. Admins PUT to approve/reject, updating `reviewed_by` and asynchronously triggering a notification to the applicant.
- **Supports Frontend:** A21, A39, W21, W39

6. Repository (`/api/repository`)

API-15: Upload & Fetch Assets

- **Method:** `GET` , `POST /api/repository`

- **Models:** `RepositoryModel`
- **Logic:** POST handles multipart file uploads (routing to AWS S3, Cloudinary, etc.) and saves URLs. GET filters results using the `access_control` arrays against the requesting user's `role_id` and `domain`.
- **Supports Frontend:** A22, A43, W22, W43

7. Chat & Real-Time (REST & WebSockets)

API-16: Chat Initialization

- **Method:** `GET /api/chat/channels` & `GET /api/chat/messages/:channel_id`
- **Models:** `ChannelModel`, `ChatModel`
- **Logic:** Fetches user's accessible channels and historical messages for pagination.
- **Supports Frontend:** A42, W42

WebSocket Architecture (Socket.io Gateway): This defines the real-time event-driven architecture for the Chat (A42/W42), Advanced Chat (A43/W43), and real-time App notifications.

WS-01: Connection & Authentication

- **Event:** `connection` (Client → Server)
- **Logic:** Authenticates socket connection using JWT token in headers. Extracts `user_id` and attaches it to the socket instance. Automatically subscribes user to their private room (e.g., `user_id_123`) for direct notifications.
- **Response:** `connected` (Server → Client)

WS-02: Join / Leave Channel (Room)

- **Event:** `join_channel` (Client → Server)
- **Payload:** `{ channel_id: string }`
- **Logic:** Server verifies if the user has access to `channel_id` (via `ChannelModel`). Subscribes the user's socket to the `channel_id` room.

- **Response:** `user_joined` (Server → Room) broadcasting `{ user_id, name, timestamp }`.

WS-03: Send Message (Core Chat & Advanced)

- **Event:** `send_message` (Client → Server)
- **Payload:** `{ channel_id: string, text: string, attachments: array }`
- **Logic:**
 1. Validates payload (text size, attachment limits).
 2. Saves the message into the `ChatModel` database.
 3. Triggers targeted push notifications via FCM/APNS if `@mentions` exist (using `NotificationModel`).
- **Response:** `receive_message` (Server → Room) broadcasting the newly saved message object to everyone in that channel in real-time.

WS-04: Typing Indicators

- **Event:** `typing_start` / `typing_stop` (Client → Server)
- **Payload:** `{ channel_id: string }`
- **Response:** `user_typing` (Server → Room) broadcasting `{ user_id, name, is_typing: boolean }` so frontend can show "User is typing..".

WS-05: Real-Time Notifications

- **Event:** Server-Side internal trigger (No Client Emit)
- **Logic:** When a task is assigned, leave is approved, or someone is mentioned, the server will emit an event directly to a specific user's private socket room.
- **Response:** `new_notification` (Server → Specific Client) broadcasting the `NotificationModel` object to trigger an in-app popup/bell ring immediately without refresh.
- **Supports Frontend:** A34, W34

8. Notifications & Community Updates (/api/notifications, /api/updates)

API-17: Fetch & Mark Read (Notifications)

- **Method:** GET, PUT /api/notifications
- **Models:** NotificationModel
- **Logic:** Fetches user's alerts. PUT marks specific or all notifications as is_read: true.
- **Supports Frontend:** A34, W34

API-18: Manage Community Updates

- **Method:** GET, POST /api/updates
- **Models:** UpdatesModel
- **Logic:** GET fetches the latest internal updates for the Dashboard. POST allows Admins to broadcast new updates.
- **Supports Frontend:** A06, W06

9. Analytics & Admin Dashboard (/api/analytics)

API-19: Analytics & Admin Dashboard

- **Method:** GET /api/analytics
- **Models:** TaskModel, UserModel, LeaveApplicationModel
- **Logic:** Runs MongoDB aggregation pipelines to calculate Dashboard graphs (Task Load Graph by status & domain) and Admin Dashboard high-level metrics (total active, alumni, etc.). Requires Admin/Manager RBAC.
- **Supports Frontend:** A06, A37, W06, W37

